



Phase 2: Deployment

Barnabas Steiner - Benjamin Hager - Jannis Weber - Lennart Esch
Tim Dorozynski - Walter Neer

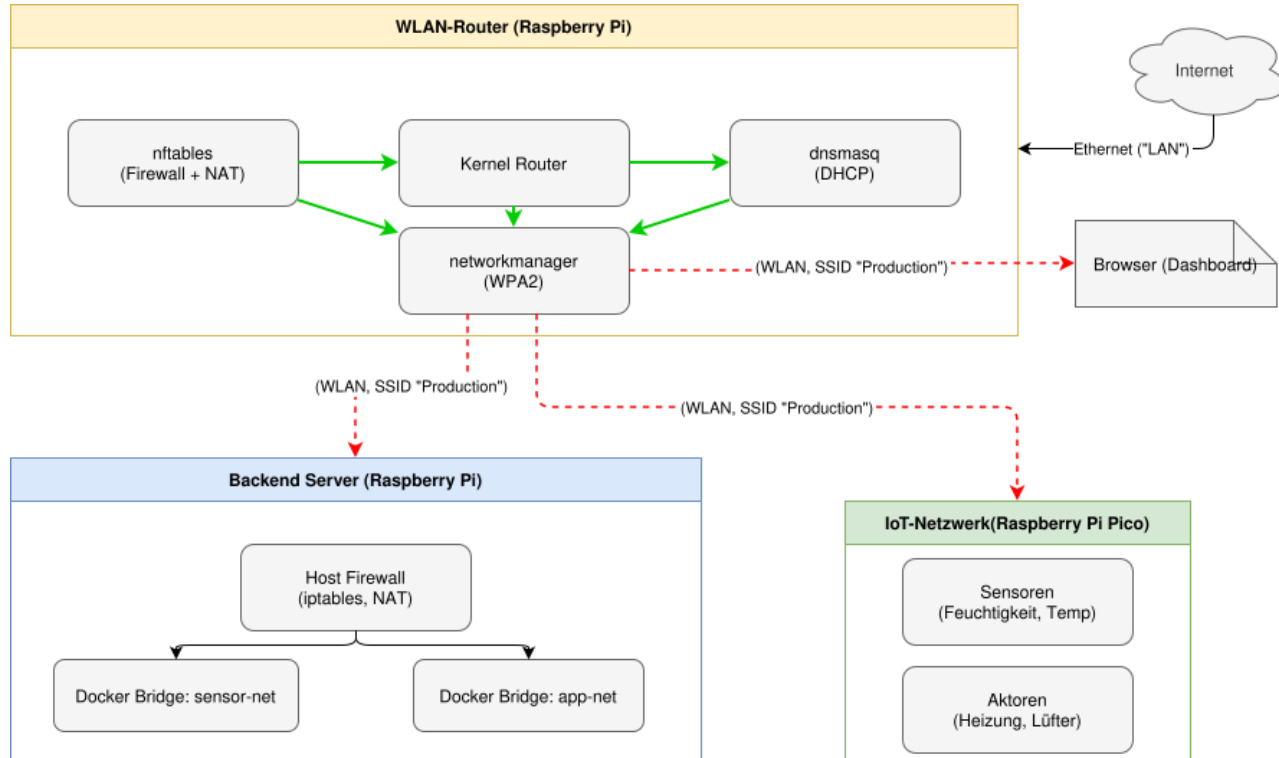
Gliederung



- Projektplan
- Deployment-Diagramm
- Bewertung der Architektur
- AP-Konfiguration
- Konfiguration der Firewall
- Integration Microcontroller
- Ergebnisstand
- Verwendete Hilfsmittel

☐	Unterelement		Owner	Status	Date	Zeitleiste ⓘ	Abhängig von ⓘ	Text
☐	Update Deployment Diag...	+	JW	Fertig		Apr. 16 - 18		
☐	Evaluate Security principl...	+		Fertig		Apr. 19 - 20	Update Deployment ...	
☐	Install OS on Pi's	+	LE +2	Fertig		Apr. 16 - 18		
☐	Setup Access Point	+	LE	Fertig		Apr. 19 - 22	Install OS on Pi's	Script: https://git...
☐	Configure Firewall	+	LE	Fertig		Apr. 23 - 25	Setup Access Point	
☐	Integrate Backend	+		Fertig		Apr. 23 - 25	Setup Access Point	
☐	Integrate Microcontroller	+		Fertig	Apr. 27	Apr. 23 - 25	Setup Access Point	
☐	+ Unterelement hinzufügen							

Deployment Diagramm



Sicherheitsprinzip-	Bewertung	Nötige Ergänzungen
Security-By-Design	• Klare Trennung von Router, Backend und IoT-Netzwerk • Kommunikationskanäle ersichtlich	
Least Common Mechanism	• Produktion hat eigenes Netzwerk • Dockerbridges zur Kommunikationsminimierung	• Darstellen von Nachrichtenqueues für Sensoren und Aktoren
Complete Mediation	• Gatekeeping durch nftables und iptables	• JWT-Token für Webnutzung hinzufügen
Least Privilege	• Trennung des Backends in app-net und sensor-net	• Hinzufügen von Benutzerrollen
Fail-Secure Defaults	• Absturz networkmanager -> sicherer Abbruch der Verbindung	• Ip-Forwarding deaktiviert, wenn Firewall nicht startet
Defence in Depth	• nftables am Router • WPA2-Verschlüsselung • iptables auf Backend-Pi • Isolierung durch Docker	

AP-Konfiguration



```
22  ▼  def configure_ap():
23      """Creates Configuration"""
24      # 1. Configuration Parameters
25      wlan_iface = "wlan0"
26      eth_iface = "eth0"
27      ssid = "Production"
28      password = "Production-01"
29      run_cmd("nmcli device set wlan0 managed no")
30
31      print(f"--- Starting Configuration on {wlan_iface} ---")
32
33      # 2. Stop services to prevent locks during config
34      print("Stopping services")
35      run_cmd("systemctl stop hostapd dnsmasq nftables")
```

AP-Konfiguration - hotsapd, dnsmasq, IP-forwarding

```
39     hostapd_conf = f"""
40     interface={wlan_iface}
41     driver=nl80211
42     ssid={ssid}
43     hw_mode=g
44     channel=7
45     wmm_enabled=1
46     macaddr_acl=0
47     auth_algs=1
48     ignore_broadcast_ssid=0
49     wpa=2
50     wpa_passphrase={password}
51     wpa_key_mgmt=WPA-PSK
52     rsn_pairwise=CCMP
53     """
```

```
58     dnsmasq_conf = f"""
59     interface={wlan_iface}
60     bind-interfaces
61     dhcp-range=192.168.4.2,192.168.4.20,255.255.255.0,24h
62     domain=wlan
63     address=/gw.wlan/192.168.4.1
64     """
```

```
74     # 6. Enable IP Forwarding (Persistent)
75     print("Enable persistent IP Forwarding")
76     run_cmd("sysctl -w net.ipv4.ip_forward=1")
77     ip_sysctl = "net.ipv4.ip_forward=1"
78     write_file("/etc/sysctl.conf", ip_sysctl)
```

Konfiguration der Firewall - Filter-Tabelle



```
86     table inet filter {
87         chain input {
88             type filter hook input priority 0;
89             policy drop;
90             iif "lo" accept
91             ct state established,related accept
92             tcp dport 22 accept
93             ip protocol icmp accept
94             iif "wlan0" udp dport { 53, 67 } accept
95         }
```

Konfiguration der Firewall - NAT-Tabelle



```
107     table ip nat {
108         chain prerouting {
109             type nat hook prerouting priority dstnat;
110             policy accept;
111         }
112         chain postrouting {
113             type nat hook postrouting priority srcnat;
114             policy accept;
115             oif "eth0" masquerade
116         }
117     }
```

Integration Microcontroller - WLAN



```
16  def connect_wifi():
17      """Verbindung zum WLAN"""
18      wlan = network.WLAN(network.STA_IF)
19      wlan.active(True)
20      wlan.connect(SSID, PASSWORD)
21
22      print("Verbinde mit WLAN...")
23      while not wlan.isconnected():
24          sleep(1)
25          print("...")
26
27      print("Verbunden! IP:", wlan.ifconfig()[0])
28      return wlan
```

SSID: "Production"

PASSWORD: "Production-01"

- Anmeldung ins WLAN als Client
- Verbindung in Schleife abwarten

Integration Microcontroller - Temperaturabfrage

- Abfrage Open-Meteo API
- Extraktion der Temperatur aus JSON
- Blinkrate durch Temperatur bestimmen:
 - `temp < 10` $\Rightarrow$ 2.0
 - `temp <= 25` $\Rightarrow$ 1.0
 - `else` $\Rightarrow$ 0.3

```
31 def fetch_outside_temperature():
32     """Daten aus Friedrichshafen"""
33     try:
34         print("Hole Außentemperatur von Open-Meteo...")
35         r = requests.get(API_URL)
36         data = r.json()
37         r.close()
38         temp = data["current_weather"]["temperature"]
39         print("Außentemperatur Friedrichshafen:", temp, "°C")
40         return temp
41     except Exception as e:
42         print("Fehler bei API:", e)
43         return None
```

Ergebnisstand



- Deploymentdiagramm für klare Übersicht der Architektur
- Produktionsnetzwerk ist funktionfähig
 - Access-Point mit routing
 - Integration von Backend und IoT durch WLAN-Clients
 - Firewall reguliert Zugriffsberechtigungen

Verwendete Hilfsmittel



monday work management: Erstellung des Projektplanes und Aufgabenzuteilung

draw.io: Erstellung des Deploymentdiagramms

github: Austausch und Versionierung von Daten